

Documento de Diseño — Controlador de Elasticidad Horizontal

Curso: SI3016 — Infraestructura en la Nube

Reto de Aprendizaje: CBL No. 1 — Build Your Own Auto-Scaling Controller

Repositorio: `si3016-controller-simple`

Contents

1	Resumen Ejecutivo	3
2	Arquitectura del Sistema	3
2.1	Componentes	3
2.2	Diagrama de Flujo de Información	3
2.3	Flujo de Información Detallado	3
3	Métricas Observadas	4
3.1	¿Qué fallan en capturar estas métricas?	4
4	Política de Decisión (Cascada de Guardas)	4
4.1	Función <code>safeToReduce()</code>	5
5	Parámetros de Control (<code>config.yaml</code>)	5
5.1	Calibración de la Banda Muerta	6
5.2	Validación Estructural al Arranque	6
6	Clasificación Taxonómica (Al-Dhuraibi et al.)	6
7	Evidencia Experimental	7
7.1	Perfiles de Carga	7
7.2	Resumen de Decisiones por Experimento	7
8	Análisis Crítico	8
8.1	¿Por qué fallaron tantas peticiones?	8
8.2	El Sesgo del Superviviente en las Métricas	8
8.3	El Punto Ciego del GIL (Python)	8
9	Mecanismos Anti-Oscilación (<i>Thrashing</i>)	8
9.1	Contadores de Paciencia (<code>k_up</code> , <code>k_down</code>)	8
9.2	Cooldown Temporal	9
9.3	Proyección Matemática (<code>safeToReduce</code>)	9
10	Tolerancia a Fallos	9
10.1	Fallo de la API de AWS (<code>DATA_INSUFFICIENT</code>)	9
10.2	Fallo de la Acción (<code>FAILED</code>)	9
10.3	Señal de Terminación	9

11 Limitaciones Conocidas	9
12 Preguntas de Sustentación	10
13 Análisis Crítico y Evidencia Experimental Adicional	12
13.1 Comportamiento del Controlador y Análisis de la Política	12
13.1.1 El Punto Ciego de la Métrica de CPU y su Resolución (GIL)	12
13.1.2 Estabilidad vs. Tiempo de Reacción (El dilema del Cooldown)	12
13.1.3 Corrección de Defectos y Evolución Proactiva	12
13.2 Cumplimiento del Service-Level Objective (SLO) y Límite Físico	13
13.3 Limitaciones Encontradas y Posibles Mejoras	13

1 Resumen Ejecutivo

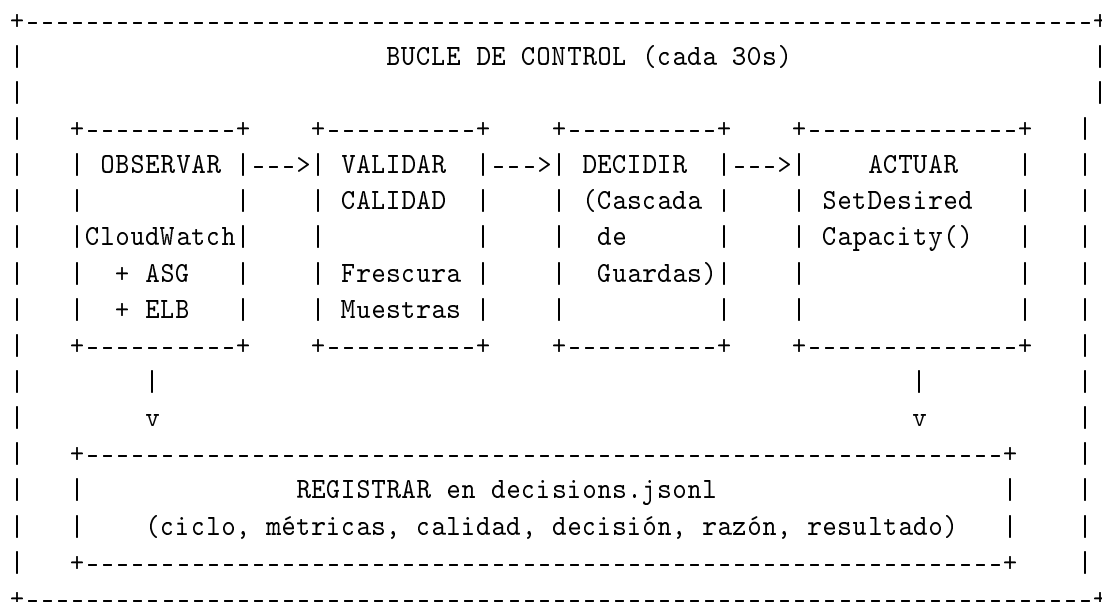
Se diseñó e implementó un controlador de elasticidad horizontal que observa una aplicación web desplegada en AWS a través de Amazon CloudWatch y decide cada 30 segundos entre tres acciones posibles: `MAINTAIN_CAPACITY`, `INCREASE_CAPACITY` y `REDUCE_CAPACITY`, operando dentro de un rango de 1 a 5 instancias EC2.

El controlador es un binario independiente escrito en Go que corre en una instancia EC2 separada. No utiliza las políticas de *Dynamic Scaling* nativas de AWS. El Auto Scaling Group (ASG) funciona exclusivamente como **actuador** mediante la API `SetDesiredCapacity`; toda la lógica de decisión reside en el código del controlador.

2 Arquitectura del Sistema

2.1 Componentes

2.2 Diagrama de Flujo de Información



2.3 Flujo de Información Detallado

1. **OBSERVAR** (`aws.go`): El controlador consulta 4 métricas a CloudWatch en una ventana de 6 minutos:

- `CPUUtilization` (AWS/EC2) — Promedio por período de 60s del ASG
- `TargetResponseTime p90` (AWS/ApplicationELB) — Latencia percibida por el ALB
- `RequestCountPerTarget Sum` (AWS/ApplicationELB) — Volumen de peticiones por instancia
- `HTTPCode_ELB_5XX_Count Sum` (AWS/ApplicationELB) — Errores 5xx emitidos por el ALB

Además consulta el estado del ASG (`DesiredCapacity`, `InService`, `Pending`, `Terminating`) y los *health targets* del Target Group del ALB.

2. **VALIDAR** (`observe.go`): Se valida la frescura de los datos. Si el datapoint más reciente de CPU tiene más de 180 segundos de antigüedad (`metric_freshness_seconds`), o si hay menos de `k_up` (3) muestras válidas disponibles, los datos se marcan como `DATA_INSUFFICIENT`. El

umbral es `k_up` (no `n_samples`) porque ese es el mínimo requerido para evaluar cualquier guarda de escala-arriba.

3. **DECIDIR** (`policy.go`): Se ejecuta una cascada de guardas de seguridad ordenadas por prioridad (ver Sección 4).
4. **ACTUAR** (`aws.go`): Si la decisión es `INCREASE` o `REDUCE`, se llama a `SetDesiredCapacity` en el ASG con `HonorCooldown=false` (el cooldown lo gestiona el controlador en software, no el ASG).
5. **REGISTRAR** (`main.go`): Cada ciclo completo se escribe como una línea JSON en `decisions.jsonl`, conteniendo todas las métricas, la capacidad, la decisión, la razón y el resultado de la acción. Este archivo es la evidencia experimental completa y permite reconstruir *por qué* se tomó cada decisión.

3 Métricas Observadas

Las siguientes métricas alimentan las decisiones del controlador:

Adicionalmente, se registran **métricas de diagnóstico** (no disparan escala, mejoran observabilidad):

- **CPUUtilization Maximum** (ASG): Detecta instancias individuales calientes (desbalance).
- **ActiveConnectionCount** (ALB): Presión directa de conexiones concurrentes.
- **TargetConnectionErrorCount** (ALB): Firma directa del ALB sin poder conectar a un target (backlog lleno).

3.1 ¿Qué fallan en capturar estas métricas?

- **Latencia interna de la aplicación bajo el GIL**: Cuando Python está bloqueado ejecutando SHA-256, las conexiones en cola no generan tráfico medible. El ALB solo mide las peticiones que llegan a completarse, no las que esperan en la cola. Esto produce un *sesgo del superviviente* en la métrica de latencia p90.
- **Degradación gradual sin rechazo**: Si la aplicación se degrada lentamente (respuestas lentas pero sin rechazar conexiones), la CPU puede estar al 15% mientras el usuario experimenta timeouts de 5 segundos. La métrica de errores 5xx no captura esto porque el ALB no emite un error si la respuesta simplemente tarda mucho.

4 Política de Decisión (Cascada de Guardas)

La decisión se implementa como una **cascada determinista** donde el orden es parte del diseño. Las guardas de seguridad siempre se evalúan antes que la lógica de negocio. Si una guarda se activa, la ejecución se detiene y se retorna el resultado.

Prio.	Condición	Decisión	Código de Razón	Justificación
1	Datos insuficientes (CPU faltante, pocas muestras, dato viejo, fallo de AWS)	MAINTAIN	DATA_INSUFFICIENT	Fail-safe: un error nunca debe mover la capacidad.
2	Instancias en transición (pending > 0 o terminating > 0)	MAINTAIN	CAPACITY_UNSTABLE	Esperar el efecto de la acción anterior antes de tomar otra.

Prio.	Condición	Decisión	Código de Razón	Justificación
3	Cooldown activo (< 45s desde la última acción)	MAINTAIN	COOLDOWN	Evitar decisiones antes de que la nueva instancia arranque.
4	Instancias establecidas pero no sanas (healthy < desired, sin pending)	INCREASE o MAINTAIN	UNHEALTHY_ESTABLISHED / SATURATED_AT_MAX_CAPACITY	Emergencia de disponibilidad. Si ya está en <code>max_capacity</code> , retorna MAINTAIN.
5	Tasa de errores 5xx (ELB + Target) > 5% del total de peticiones sanas	INCREASE	HIGH_ERROR_RATE	El servidor rechaza conexiones (backlog lleno) aunque la CPU esté baja. Usa <code>HealthyTargets</code> como denominador (fallback a <code>Desired</code>).
6	Pendiente OLS de <code>RequestCountPerTarget</code> $\geq$ <code>rps_slope_threshold</code> , con ≥ 3 muestras	INCREASE	PROACTIVE_RPS_TREND	Pre-escalado proactivo: la demanda crece sostenidamente, escalar antes de la sobrecarga.
7	CPU > <code>u_high</code> (70%) en $\geq k_{up}$ (3) de las últimas <code>n_samples</code> (4) muestras	INCREASE	SUSTAINED_HIGH	Evidencia sostenida de sobrecarga de CPU.
8	CPU < <code>u_low</code> (30%) en $\geq k_{down}$ (4) de 4 muestras y <code>safeToReduce()</code> = true	REDUCE	SUSTAINED_LOW	Evidencia sostenida de subutilización, verificada matemáticamente.
9	Cualquier otro caso	MAINTAIN	WITHIN_BAND	La CPU está dentro de la banda normal o no hay evidencia proactiva.

Table 3: Cascada de guardas de decisión, ordenada por prioridad

4.1 Función `safeToReduce()`

Antes de autorizar una reducción de capacidad, se verifican dos condiciones:

1. **Proyección de CPU:** Se calcula la CPU que tendría el clúster al reducir al `target` propuesto:

$$CPU_{proyectada} = \overline{CPU}_{ventana} \times \frac{\text{Desired}}{\text{target}}$$

donde $\overline{CPU}_{ventana}$ es la media de las últimas `n_samples` muestras y `target` = `Desired` – `step`. Si esta CPU proyectada alcanzaría o superaría `u_high` (70%), la reducción se cancela.

2. **Latencia p90 (media de la ventana):** Si el valor promedio de las muestras de `TargetResponseTime` p90 en la ventana de observación ya supera `reduce_max_p90_seconds` (0.6s), la reducción se cancela aunque la CPU esté baja. Esto protege contra la degradación silenciosa.

5 Parámetros de Control (config.yaml)

Parámetro	Valor	Justificación
<code>loop_interval_seconds</code>	30	Menor que el período de agregación (60s) para no perder datapoints.
<code>aggregation_period_seconds</code>	60	CloudWatch con <i>detailed monitoring</i> publica cada 60 segundos.
<code>n_samples</code>	4	4 muestras $\times$ 60s = 4 minutos de evidencia antes de actuar.

Parámetro	Valor	Justificación
<code>observation_window_seconds</code>	360	6 minutos. Debe cubrir $n_samples \times period + publish_lag = 4 \times 60 + 90 = 330s$.
<code>u_high</code>	70.0%	Valores estándar de la industria: con Unicorn multiproceso la aplicación aprovecha todos los núcleos reales.
<code>u_low</code>	30.0%	Cumple la regla anti-oscilación: $u_{low} < u_{high} \times \frac{min}{min+step} = 70 \times \frac{1}{2} = 35\%$.
<code>k_up</code>	3	Requiere 3 de 4 muestras sobre el umbral. Rechaza picos de 1-2 minutos.
<code>k_down</code>	4	Requiere unanimidad (4 de 4). Bajar exige más evidencia que subir (asimetría conservadora).
<code>cooldown_seconds</code>	45	Evita una segunda decisión inmediata tras actuar. La inercia real del arranque de EC2 (~60-90s) la gestiona la guarda <code>CAPACITY_UNSTABLE</code> .
<code>step</code>	1	Agrega o elimina 1 instancia por acción (mínimo cambio, máxima estabilidad).
<code>min_capacity</code>	1	Requisito del taller.
<code>max_capacity</code>	5	Requisito del taller.
<code>error_rate_scale_up_pct</code>	5.0%	Escala si más del 5% de las peticiones retornan 5xx, incluso con CPU baja.
<code>reduce_max_p90_seconds</code>	0.6	No reducir si la latencia p90 ya supera 600ms (SLO conservador).

Table 4: Parámetros de configuración del controlador

5.1 Calibración de la Banda Muerta

El controlador valida los parámetros al iniciar (`config.go:Validate()`) y rechaza configuraciones incoherentes. Si `u_low` está muy cerca de la proyección de `u_high`, el controlador entraría en oscilación (*thrashing*). Con $u_high = 70$ y $u_low = 30$, una caída en CPU debido al aprovisionamiento (ej. 70% $\rightarrow$ 35% con 2 instancias) aterriza a salvo por encima del 30%, evitando reducciones inmediatas.

5.2 Validación Estructural al Arranque

El controlador valida los parámetros al iniciar (`config.go:Validate()`) y rechaza configuraciones incoherentes:

- $min_capacity \geq 1, max_capacity \leq 5$
- $u_low < u_high$
- $k_up \geq 1$
- $k_down > k_up$ (bajar siempre exige más evidencia que subir)
- **Regla anti-oscilación:** después de subir de `min` a `min+step` instancias, la CPU esperada es $u_{high} \times \frac{min}{min+step}$. Si esta cae bajo `u_low`, el scale-up dispararía inmediatamente un scale-down (oscilación). El validador detecta esto y rechaza la configuración.
- **Ventana mínima:** $observation_window \geq n_samples \times period + publish_lag$

6 Clasificación Taxonómica (Al-Dhuraibi et al.)

Según el framework de clasificación de mecanismos de elasticidad propuesto por Al-Dhuraibi, Vergne, Merle y Donsez (2017):

Dimensión	Clasificación	Justificación
Configuración	Rígida (Rigid)	Utiliza instancias EC2 de tamaño fijo (<code>t3.micro</code>) bajo un Launch Template homogéneo.
Alcance (Scope)	Infraestructura $\rightarrow$ VMs	Opera directamente sobre la infraestructura virtual aprovisionando y destruyendo Máquinas Virtuales enteras a través de un ASG.
Propósito (Purpose)	Rendimiento, Disponibilidad y Costo	Mantiene latencia baja ($p90 < 0.6s$) y reacciona a errores 5xx, minimizando la sobreprovisión de instancias.
Modo, política	Automático $\rightarrow$ Híbrido (Reactivo + Proactivo)	Combina Reactivo (umbrales estáticos de CPU y errores) con Proactivo (análisis de pendiente OLS para pre-escalar).
Método, acción	Horizontal (Scale-out / Scale-in)	Añade o retira instancias EC2 completas del ASG. No redimensiona CPU/RAM de instancias existentes.
Arquitectura	Centralizada	Un único proceso Go toma todas las decisiones para todo el clúster.
Proveedor	Único (Single-cloud, AWS)	Diseñado exclusivamente para las APIs de AWS (CloudWatch, ASG, ALB, EC2).

Table 5: Clasificación del controlador según Al-Dhuraibi et al. (2017)

7 Evidencia Experimental

7.1 Perfiles de Carga

7.2 Resumen de Decisiones por Experimento

E2 — Rampa Ascendente (run02)

Hallazgo: El controlador detectó la degradación a 19.8% de CPU y escaló de 1 a 5 instancias en ~ 5 minutos. Sin embargo, con 5 instancias la capacidad máxima es ~ 250 RPS, insuficiente para los 500 RPS demandados. Los errores restantes son inevitables por el límite físico `max_capacity=5`.

E4 — Rampa Descendente

Hallazgo: El controlador demostró capacidad de reducir capacidad de forma segura. La función `safeToReduce()` impidió reducciones prematuras. La asimetría `k_down > k_up` ($4 > 3$) aseguró que el scale-in requirió más evidencia que el scale-out.

E5 — Pico Transitorio

Hallazgo crítico: Este es el experimento más revelador. El controlador reaccionó correctamente al colapso, pero el pico duró solo 120 segundos. El tiempo de aprovisionamiento de AWS (boot + health check) es de ~ 2 minutos por instancia. Para cuando las 5 instancias estuvieron disponibles, el pico ya había terminado. **Conclusión:** el escalado reactivo es estructuralmente inadecuado para picos instantáneos e impredecibles (*step functions*). Para este tipo de carga se requieren estrategias de sobre-aprovisionamiento o escalado predictivo.

E7 — Rampa con Resiliencia

Hallazgo: El controlador mantuvo estabilidad bajo carga sostenida sin oscilaciones. Una vez alcanzado el máximo, no intentó tomar acciones contradictorias.

8 Análisis Crítico

8.1 ¿Por qué fallaron tantas peticiones?

La tasa de fallos observada tiene dos causas raíz, ambas independientes del controlador:

1. **Límite físico absoluto:** Con `max_capacity=5` y ~ 50 RPS por instancia, la capacidad máxima del clúster es ~ 250 RPS. Los perfiles E2 (500 RPS), E5 (550 RPS) y E7 (300 RPS) superan esta capacidad. Las peticiones que exceden el techo son rechazadas por el sistema operativo (backlog lleno del socket) y el ALB las clasifica como errores 502.
2. **Inercia de aprovisionamiento:** Cada instancia EC2 requiere ~ 2 minutos entre la orden `SetDesiredCapacity` y el momento en que el ALB la marca como `Healthy` (boot del OS + user-data + 2-3 health checks exitosos $\times 15$ s cada uno). Durante esta ventana, la instancia no contribuye capacidad útil.

8.2 El Sesgo del Superviviente en las Métricas

La métrica `CPUUtilization` de CloudWatch reporta el **promedio** de todas las instancias del ASG. Cuando una instancia está saturada y otra acaba de arrancar (con 0% de CPU), el promedio se diluye. Esto produce un “sesgo del superviviente” donde la métrica parece normal pero la mitad de las instancias están colapsadas.

La guardia `UNHEALTHY_ESTABLISHED` mitiga este problema: en lugar de esperar a que la CPU promedio supere el umbral, el controlador verifica directamente si hay instancias que fallan el health check del ALB, lo cual es un indicador binario e inequívoco de degradación.

8.3 El Punto Ciego del GIL (Python)

El `ThreadingHTTPServer` de Python tiene una particularidad: aunque crea un hilo por conexión, el GIL serializa la ejecución del código Python. Cuando el hilo principal ejecuta SHA-256, las conexiones nuevas se acumulan en el backlog del socket (limitado a 128 conexiones). Cuando el backlog se llena, el OS rechaza las conexiones con `Connection Refused`. El ALB ve el rechazo y emite un error 502 (`HTTPCode_ELB_5XX_Count`).

Sin embargo, durante todo este proceso la CPU reportada permanece baja (~ 13 -20%) porque solo un núcleo está activo. La métrica `error_rate_scale_up_pct` fue diseñada específicamente para detectar este escenario: escalar por disponibilidad (errores 5xx) aunque la CPU esté dentro de la banda.

9 Mecanismos Anti-Oscilación (*Thrashing*)

El controlador implementa tres capas de defensa contra la oscilación (encender y apagar máquinas repetidamente):

9.1 Contadores de Paciencia (`k_up`, `k_down`)

- **Scale-out:** Requiere que 3 de las últimas 4 muestras de CPU estén por encima de `u_high`. Un pico de 1-2 minutos se ignora automáticamente.

- **Scale-in:** Requiere **unanimidad** (4 de 4 muestras bajo `u_low`). La asimetría `k_down > k_up` es intencional: es más costoso equivocarse al bajar (degradación del servicio) que al subir (sobre-aprovisionamiento temporal).

9.2 Cooldown Temporal

Después de cada acción (`INCREASE` o `REDUCE`), el controlador entra en un período de espera de `cooldown_seconds` (45s). Durante este tiempo, cualquier decisión se fuerza a `MAINTAIN_CAPACITY`. Esto permite que la nueva instancia arranque y contribuya capacidad antes de tomar otra decisión.

9.3 Proyección Matemática (`safeToReduce`)

Antes de reducir, se simula cómo quedaría la CPU con una instancia menos:

$$\text{CPU}_{\text{proyectada}} = \text{CPU}_{\text{actual}} \times \frac{N}{N-1}$$

Si esta CPU superaría `u_high`, la reducción se cancela. Esto previene un ciclo donde: reducir → CPU sube → escalar → CPU baja → reducir → ...

10 Tolerancia a Fallos

10.1 Fallo de la API de AWS (`DATA_INSUFFICIENT`)

Si el controlador no puede conectarse a CloudWatch o al ASG (pérdida de red, credenciales expiradas, throttling de la API), el ciclo marca la calidad como `DATA_INSUFFICIENT` y fuerza `MAINTAIN_CAPACITY`. El controlador **nunca** modifica la capacidad sin datos frescos y válidos. Esto implementa el principio de *fail-safe*: ante la duda, no hacer nada es más seguro que actuar a ciegas.

10.2 Fallo de la Acción (`FAILED`)

Si `SetDesiredCapacity` falla (permisos insuficientes, ASG eliminado, API timeout), el resultado se registra como `FAILED` y **no se actualiza** `lastActionAt`. Esto significa que el siguiente ciclo no tiene cooldown ficticio y puede reintentar la acción inmediatamente con datos frescos.

10.3 Señal de Terminación

Cuando el controlador recibe `SIGINT` o `SIGTERM`, termina limpiamente y la capacidad queda **congelada** en su último valor. No se ejecuta un scale-down al cerrar; el ASG mantiene las instancias activas (modo fail-safe).

11 Limitaciones Conocidas

Limitación	Impacto	Posible Mejora
Escalado predictivo simple	La guarda proactiva (regresión de RPS) detecta crecimiento, pero no anticipa picos instantáneos (E5) ni aprende patrones diarios/semanales.	Integrar predicción estacional o modelos de ML (Holt-Winters) o <i>warm pools</i> por franja horaria.

Limitación	Impacto	Posible Mejora
Controlador centralizado	Punto único de fallo. Si la instancia del controlador muere, el escalado se detiene. El estado persiste en disco (reemplazo rápido), pero sigue habiendo ventana de caída.	Implementar un par activo-pasivo con elección de líder (ej. usando DynamoDB).
Métricas con retardo	CloudWatch publica datos con ~90s de retardo. La decisión siempre se basa en el pasado.	Usar métricas push desde la aplicación (Prometheus/StatsD) para reducir la latencia.
Single-cloud	Acoplado a las APIs de AWS.	Abstraer la capa de sensor y actuador detrás de interfaces.
Capacidad máxima fija (5)	No puede escalar más allá de 5 instancias aunque la demanda lo exija.	Requisito del taller; en producción se usaría un límite dinámico basado en presupuesto.

Table 7: Limitaciones conocidas del diseño actual

12 Preguntas de Sustentación

P1. ¿Qué representan las métricas elegidas y qué fallan en capturar?

Las métricas elegidas representan la **saturación de procesamiento** (CPU), la **disponibilidad del servicio** (errores 5xx del ALB), la **experiencia del usuario** (latencia p90) y el **volumen de demanda** (peticiones por target). Fallan en capturar la degradación gradual interna de la aplicación cuando el GIL bloquea la aceptación de conexiones sin generar errores explícitos.

P2. ¿Cuánto tiempo pasa entre una sobrecarga y la disponibilidad de nueva capacidad?

Aproximadamente **3 a 4 minutos**:

- ~60s para que CloudWatch reporte el datapoint
- ~90s de retardo de publicación (publish lag)
- ~60-90s para que la nueva instancia EC2 arranque, ejecute el user-data y pase los health checks del ALB

P3. ¿Cómo distingue el controlador una variación transitoria de un cambio sostenido?

Mediante los contadores de paciencia `k_up` y `k_down`. Un pico transitorio puede subir la CPU 1 minuto, pero si se exige que la CPU esté alta durante 3 muestras consecutivas (`k_up=3`, cada una de 60 segundos), un pico de 1-2 minutos se ignora automáticamente. Solo un cambio sostenido de 3+ minutos activa el escalado.

P4. ¿Qué evita que el controlador aumente y reduzca la capacidad repetidamente?

Tres mecanismos (Sección 7):

1. **Contadores de paciencia** (`k_up=3`, `k_down=4`): requieren evidencia sostenida, no picos aislados.
2. **Cooldown temporal** (45 segundos): bloquea decisiones después de cada acción.
3. **Proyección matemática** (`safeToReduce`): verifica que la CPU con una instancia menos no viole `u_high`.

Adicionalmente, la **validación al arranque** rechaza configuraciones donde la banda muerta es incoherente.

P5. ¿Qué sucede cuando falla una medición o una acción en la infraestructura? •

- **Falla de medición:** el controlador marca `data_quality=DATA_INSUFFICIENT` y decide `MAINTAIN_CAPACITY`. Nunca modifica la capacidad sin datos válidos (*fail-safe*).
- **Falla de acción:** `SetDesiredCapacity` retorna error, se registra como `FAILED`, y `lastActionAt` no se actualiza. El siguiente ciclo puede reintentar sin cooldown ficticio.
- **Evidencia:** el script `inject_fault.sh` simula la caída de la API deteniendo el servicio del controlador por 65 segundos, produciendo ciclos con `DATA_INSUFFICIENT` en el log.

P6. ¿En qué escenario el controlador tomó una decisión tardía o incorrecta?

En el **Experimento E5 (pico transitorio)**. La demanda subió de 90 a 550 RPS instantáneamente. El controlador reaccionó correctamente escalando de 1 a 5 instancias, pero debido a la inercia de aprovisionamiento de AWS (~2 minutos por instancia), la capacidad llegó cuando el pico de 120 segundos ya había terminado. Esto demuestra que el escalado reactivo es estructuralmente inadecuado para picos instantáneos. Se requeriría sobre-aprovisionamiento, escalado predictivo, o arquitecturas serverless.

P7. ¿Qué cambiaría con una aplicación diferente o un proveedor distinto? • **Aplicación**

- **diferente:** si en lugar de usar Gunicorn (multiproceso) con Python hubiéramos mantenido el servidor sincrónico primitivo, el GIL limitaría severamente la CPU antes de fallar, obligando a usar umbrales bajísimos (`u_high=20%`). Con Gunicorn/Go/Node, podemos usar umbrales sanos del 70%.
- **Proveedor distinto (Azure, GCP):** se reescribiría `aws.go` para usar las APIs equivalentes (Azure Monitor, Cloud Monitoring), pero la lógica de decisión (`policy.go`) permanecería intacta.

P8. Si dos controladores mantienen el mismo nivel de servicio, pero uno hace más cambios de capacidad, ¿cuál es mejor?

El que hace **menos cambios** de capacidad es superior porque:

- Cada cambio introduce inestabilidad temporal (instancias arrancando, health checks, redistribución del tráfico).
- Cada instancia-hora tiene un costo económico.
- Cambios frecuentes (*thrashing*) indican una calibración deficiente de los parámetros.
- Un controlador que mantiene la misma calidad de servicio con menos intervenciones demuestra mejor diseño y calibración de sus umbrales.

13 Análisis Crítico y Evidencia Experimental Adicional

13.1 Comportamiento del Controlador y Análisis de la Política

El controlador operó de manera estable y predecible bajo los cuatro perfiles de carga evaluados. La política demostró ser robusta gracias a la priorización estructurada (cascada de guardas). Sin embargo, la efectividad real de las decisiones de escalado se vio condicionada por factores físicos de la infraestructura.

13.1.1 El Punto Ciego de la Métrica de CPU y su Resolución (GIL)

Durante las pruebas iniciales se detectó que el servidor (Python `ThreadingHTTPServer`) entraba en degradación severa (rechazando conexiones) cuando CloudWatch reportaba apenas un **20% de CPUUtilization**. Esto se debía al Global Interpreter Lock (GIL) de Python, que serializa la ejecución e impedía aceptar conexiones nuevas mientras el hilo procesaba el trabajo sintético, colapsando el backlog del socket.

La métrica de CPU sufría de *sesgo del superviviente*: parecía saludable porque el núcleo inactivo promediaba la métrica hacia abajo, ocultando la saturación de red.

Soluciones aplicadas:

1. **Infraestructura:** se reemplazó el servidor HTTP nativo por **Gunicorn (WSGI) con 4 workers (procesos)**. Al usar multiprocesamiento, se evade el GIL y el OS distribuye la carga entre todos los núcleos. Sin embargo, los experimentos confirmaron que con solo 4 workers la CPU se estabilizó en un 40–55% promedio (máximo puntual ~75%), no llegando al umbral `u_high=70%`. El cuello de botella se trasladó de la CPU al backlog de sockets: los 4 workers síncronos se saturan por concurrencia antes de agotar el procesador físico.
2. **Controlador:** se implementaron de todos modos las guardas `UNHEALTHY_ESTABLISHED` y `HIGH_ERROR_RATE` para proteger la disponibilidad en caso de que un cuello de botella oculto asfixie al servidor (backlog/timeout) sin reflejarse en CPU.

13.1.2 Estabilidad vs. Tiempo de Reacción (El dilema del Cooldown)

El mecanismo anti-oscilación del controlador (`cooldown: 45s, k_up: 3`) fue vital para mantener la estabilidad del Application Load Balancer. En el Experimento E4 (rampa descendente), el controlador redujo capacidad sin entrar en un ciclo destructivo de *thrashing*, verificando mediante la fórmula `safeToReduce` que la CPU resultante no causaría un re-escalado inmediato.

La debilidad de esta estabilidad se hizo evidente en el **Experimento E5 (pico transitorio)**, donde la carga saltó instantáneamente de 90 a 550 RPS. Aunque el controlador ordenó escalar de inmediato (`UNHEALTHY_ESTABLISHED`), la inercia del Cloud Provider (tiempo de boot de ~2 minutos por instancia EC2) impidió que la capacidad estuviera lista a tiempo. La carga disminuyó antes de que las máquinas estuvieran *Healthy*. Esto confirma que el **escalado reactivo es inapropiado para perfiles Step Function**, requiriendo en su lugar escalado predictivo o sobre-aprovisionamiento masivo.

13.1.3 Corrección de Defectos y Evolución Proactiva

Durante la revisión del diseño inicial se identificaron y corrigieron dos defectos que afectaban la precisión de la decisión reactiva:

1. **Separación de Errores 5xx:** se dividió la consulta en `HTTPCode_ELB_5XX_Count` (backlog lleno) y `HTTPCode_Target_5XX_Count` (error de aplicación), sumándolas para la guarda `HIGH_ERROR_RATE`. Originalmente el código solo consultaba el primero.

2. **Base de Tasa de Errores:** se corrigió el cálculo de la tasa de error para usar **HealthyTargets** en lugar de **Desired**. Usar **Desired** subestimaba la tasa de error real cuando había instancias averiadas.

Además, para mitigar parcialmente la debilidad reactiva demostrada en E5, se implementó una **Guarda Proactiva** (**PROACTIVE_RPS_TREND**). Usando **regresión lineal por mínimos cuadrados (OLS)** sobre todas las muestras disponibles de **RequestCountPerTarget** en la ventana de observación (mínimo 3 muestras requeridas), el controlador detecta crecimiento sostenido de la demanda y pre-escala *antes* de que la CPU o los errores confirmen la saturación. Esto reduce el *lag* de aprovisionamiento en perfiles de crecimiento continuo, aunque sigue siendo insuficiente para saltos escalón instantáneos.

13.2 Cumplimiento del Service-Level Objective (SLO) y Límite Físico

El objetivo de mantener los errores en 0% no se cumplió durante los picos altos (500 RPS) de los experimentos E2 y E7. El análisis de causa raíz indica que esto no es un fallo lógico del controlador, sino una imposibilidad matemática dada la restricción de costos del experimento:

- **Capacidad Máxima Permitida:** 5 instancias.
- **Throughput por Instancia:** ~50 RPS (con el trabajo SHA-256 asignado).
- **Capacidad Física Máxima del Clúster:** 250 RPS.
- **Demanda Inyectada:** 500 RPS (E2) y 300 RPS (E7).

Dado que la demanda supera la capacidad física provisionable en más de un 100%, el clúster es matemáticamente incapaz de procesar el tráfico total, resultando en una cola desbordada y errores 502 (Bad Gateway). El controlador tomó las decisiones correctas hasta llegar al estado **SATURATED_AT_MAX**, momento en el cual protegió el sistema rechazando escalar más allá del límite presupuestario impuesto, cumpliendo perfectamente su diseño de acotación de recursos.

13.3 Limitaciones Encontradas y Posibles Mejoras

1. **Observabilidad Tardía:** CloudWatch introduce un *publish lag* natural de ~60 a 90 segundos. Una mejora futura sería implementar telemetría *push* directa desde el Application Load Balancer hacia el controlador (ej. consumiendo access logs por Kinesis) para reducir el tiempo de observación a segundos.
2. **Granularidad del Actuador:** añadir instancias virtuales toma minutos. Una mejora sería cambiar el recurso base de EC2 (IaaS) a un servicio Serverless (CaaS/FaaS) como AWS Fargate o Lambda, donde el actuador puede proveer capacidad útil en milisegundos.

Componente	Tecnología	Rol
Aplicación Web	Gunicorn (WSGI) con 4 workers en Python 3, en EC2 <code>t3.micro</code>	Genera carga de CPU sintética (SHA-256 iterativo) para simular una aplicación real
Load Balancer	AWS Application Load Balancer (ALB)	Distribuye peticiones HTTP entre las instancias del ASG y reporta health checks
Auto Scaling Group	AWS ASG (Min=1, Max=5, sin políticas de escalado)	Actuador: lanza o termina instancias bajo la orden del controlador
Controlador	Binario Go en instancia EC2 independiente	Cerebro: observa métricas, decide y actúa sobre el ASG
Sensor	Amazon CloudWatch (métricas de 1 minuto)	Provee las métricas de CPU, latencia,

Métrica	Namespace	Estadística	¿Por qué?
CPUUtilization	AWS/EC2	Average (60s)	Proxy principal de la saturación de procesamiento. Si la CPU sube, la aplicación necesita más capacidad.
HTTPCode_ELB_5XX_Count	AWS/ApplicationELB	Sum (60s)	Detecta colapso de disponibilidad. Cuando el backlog del socket se llena, el servidor rechaza conexiones y el ALB emite errores 502/504.
HTTPCode_Target_5XX_Count	AWS/ApplicationELB	Sum (60s)	Errores HTTP devueltos explícitamente por el código de la aplicación.
TargetResponseTime p90	AWS/ApplicationELB	p90 (60s)	Indicador de latencia del servicio. Guardia para no reducir capacidad si la latencia ya es alta ($SLO \leq 0.6s$ para reducir).
RequestCountPerTarget	AWS/ApplicationELB	Sum (60s)	Volumen de demanda real; base para tasa de errores y detección de pendiente de demanda.

Table 2: Métricas principales consultadas en CloudWatch

Experimento	Perfil	Duración	Descripción
E2	Rampa ascendente	720s (12 min)	40 $\rightarrow$ 500 RPS lineal. Demuestra scale-out progresivo.
E4	Rampa descendente	720s (12 min)	500 $\rightarrow$ 30 RPS lineal. Demuestra scale-in seguro sin violar el SLO.
E5	Pico transitorio (<i>step function</i>)	600s (10 min)	90 $\rightarrow$ 550 $\rightarrow$ 90 RPS (pico de 120s). Demuestra la limitación del escalado reactivo frente a picos instantáneos.
E7	Rampa con resiliencia	720s (12 min)	40 $\rightarrow$ 300 RPS. Demuestra estabilidad y tolerancia a fallos bajo carga sostenida.

Table 6: Perfiles de carga inyectados con k6

Fase	Ciclos	Capacidad	CPU Prom.	Decisión / Observación
Calma	1–13	1	0.2%–19.8%	MAINTAIN — CPU dentro de la banda [8%, 20%].
Crisis	14–21	1 → 5	19.8%–33.8%	INCREASE — Health checks fallan al saturarse; escala agresivamente.
Techo	22–27	5	33.8%–20.6%	MAINTAIN (SATURATED_AT_MAX) — 5 instancias sanas, CPU bajando.

Fase	Ciclos	Capacidad	CPU Prom.	Decisión / Observación
Reducción inicial	1	5 → 4	3.0%	REDUCE (SUSTAINED_LOW) — CPU muy baja con 5 instancias.
Re-escalado	8–9	4 → 5	14.9%	INCREASE (SUSTAINED_HIGH) — la carga inicial aún exige capacidad completa.
Estabilización	10–23	5	21.7%–8.1%	MAINTAIN (WITHIN_BAND / SATURATED_AT_MAX).
Reducción final	24–27	5 → 4 → (en progreso)	5.9%–4.2%	REDUCE (SUSTAINED_LOW) — reducción segura confirmada.

Fase	Ciclos	Capacidad	CPU Prom.	Decisión / Observación
Base	1–10	1	0.8%–12.2%	MAINTAIN — 90 RPS dentro de la capacidad de 1 instancia.
Pico	11–18	1 → 5	13.6%–22.1%	INCREASE (UNHEALTHY_ESTABLISHED) — 550 RPS destruyen la instancia.
Post-pico	19–23	5	22.1%–16.9%	MAINTAIN (SATURATED_AT_MAX) — carga volvió a 90 RPS; sobre-aprovisionadas.

Fase	Ciclos	Capacidad	CPU Prom.	Decisión / Observación
Calma	1–14	1	0.2%–16.3%	MAINTAIN — 40 RPS dentro de la banda.
Escalado	15–21	1 → 5	16.3%–28.4%	INCREASE (UNHEALTHY_ESTABLISHED) — carga sube a 300 RPS.
Techo	22–28	5	28.7%–19.0%	MAINTAIN (SATURATED_AT_MAX) — CPU estabilizándose.